Stigmergic-Quorum Admission Control: A Practitioner's Guide

`moderation-pipeline-prototype`*

August 2026

1 When you need this

All three conditions together:

1. Your system has a cheap, tractable fast path and an expensive, worst-case-exponential (or otherwise costly) fallback path, and cases fall into the fallback whenever the fast path returns something uninformative.
2. Admission to the fallback is currently gated by nothing more than "has this case received at least one confirming signal" — the first plausible-looking report, argument, or vote lets it through.
3. An adversary can manufacture that one confirming signal cheaply, in bulk, in a single burst, without needing to sustain anything or coordinate identities over time.

The reference deployment is a four-stage content-moderation pipeline whose Stage 2b falls back to NP-complete preferred-extension enumeration in an abstract argumentation framework (Dung, 1995) when the grounded extension is empty and a case's escalation timeout has expired. Nothing below is specific to argumentation frameworks, though: the gate treats the expensive stage as an opaque resource pool with a release signal, and slots into any admission boundary matching the three conditions above (job queues that fall back to an exhaustive solver, moderation escalations to human review, rate-limited access to an expensive model call).

What you get. A burst of any size, in a single tick, from any number of distinct or shared identities, is blocked unconditionally. A sustained, patient, identity-diverse attacker is not blocked, but is delayed by roughly a factor of five under the reference tuning — §4 shows how that factor is a dial, not a constant, and what it costs to turn it.

2 The gate in three rules

1. **Decay every tick, before anything else.** Each case carries a score that shrinks by a fixed fraction ρ every discrete time step, whether or not the case received any reinforcement that step. This is the ordinary pheromone-evaporation update ($\tau \leftarrow (1-\rho)\tau$) from ant colony optimization, applied to an admission-pressure score instead of a trail strength. Decay runs *first*, so a case that goes quiet starts losing ground immediately, not after some grace period.

*Companion to the paper *A Decay-Weighted, Source-Independence-Discounted, Load-Adaptive Admission Gate for NP-Complete Argumentation Stages* (same package), which carries the formal model, the two propositions and their proofs, the calibration-tension corollary, and the prior-art survey. This guide carries the applied material: walkthrough, tuning procedure, troubleshooting, adaptation.

2. **Cap the tick's contribution, and discount repeats from the same source.** Within one tick, dedupe the reinforcing sources first (a source submitting five items in one tick is one reinforcement event, not five), then sum: 1.0 per source seen for the first time in the last w ticks, a small fraction δ per source seen again inside that window. Cap the sum at a fixed ceiling γ *after* the dedupe-and-discount step, never before. Getting this order wrong is the single most consequential implementation mistake in this whole gate — §6 covers it as the first troubleshooting entry, because it happened here.

3. **Raise the bar when the expensive tier is busy.** The admission threshold is not a constant; it is $\theta_0(1 + \kappa L/C)$, where L is the number of cases currently occupying the expensive tier, system-wide, right now — not a cumulative count of everything ever admitted. Getting *that* distinction wrong is the second troubleshooting entry.

3 Annotated walkthrough

The reference implementation is `src/admission_control.py`, class `StigmergicQuorumAdmissionControl`. Everything below is that file, in reading order, with the reasoning moved out of the comments and into prose.

3.1 The tunables and their defaults

```
DEFAULT_DECAY_RATE = 0.10           # rho
DEFAULT_TICK_CAP = 1.0              # gamma
DEFAULT_INDEPENDENCE_WINDOW = 3     # w, in ticks
DEFAULT_REPEAT_DISCOUNT = 0.05      # delta
DEFAULT_BASE_THRESHOLD = 4.0        # theta_0
DEFAULT_LOAD_SENSITIVITY = 0.5      # kappa
DEFAULT_CAPACITY = 5                # C
```

These seven numbers are not independent knobs to be tuned one at a time against a vibe. The paper's Corollary 1 (the calibration tension) proves that the ratio $x = \theta_0\rho/\gamma$, evaluated at zero load, governs *both* how easy a genuine, moderately-corroborated dispute is to admit *and* how many sustained ticks a fully saturated attacker needs. There is no way to move x that helps one side without hurting the other. The reference values above were chosen by measuring that trade-off directly (not tuned backward from a desired test result): they require an attacker to sustain roughly five ticks of full-saturation effort, while keeping the bar clearable for a dispute that gets an independent reinforcement on the order of 40–50% of ticks on average. If your deployment's genuine-dispute reinforcement rate is different, re-derive x for your population before touching any single constant (§4).

3.2 The per-case state and the tick loop

```
@dataclass
class CaseAdmissionState:
    score: float = 0.0
    last_reinforced_tick: dict[str, int] = field(default_factory=dict)
    admitted: bool = False
    admitted_tick: int | None = None
```

`last_reinforced_tick` is keyed by cohort id, not by event: it is the mechanism's entire memory of who has recently reinforced this case, and it is what makes the independence discount possible without storing full event history.

```
def tick(self, t: int, reinforcements: dict[str, list[str]]) -> set[str]:
    newly_admitted: set[str] = set()
    touched = set(self._cases) | set(reinforcements)
    for case_id in touched:
        state = self._cases.setdefault(case_id, CaseAdmissionState())
        if state.admitted:
            continue
        # (1) decay first
        state.score *= (1 - self.decay_rate)
        # (2) accumulate this tick's reinforcement, deduped, weighted,
        #     capped
        contrib = 0.0
        for cohort in set(reinforcements.get(case_id, [])):
            last = state.last_reinforced_tick.get(cohort)
            fresh = last is None or (t - last) >= self.independence_window
            contrib += 1.0 if fresh else self.repeat_discount
            state.last_reinforced_tick[cohort] = t
        contrib = min(contrib, self.tick_cap)
        state.score += contrib
        # (3) admit only if the load-adaptive threshold is crossed
        if state.score >= self.current_threshold():
            state.admitted = True
            state.admitted_tick = t
            self._pending_admitted.add(case_id)
            newly_admitted.add(case_id)
    return newly_admitted
```

Notice `set(reinforcements.get(case_id, []))` on the cohort loop — that `set()` call is the dedupe step, and it runs before the discount logic, not after. This is the fix for the first bug in §6; if you are adapting this gate to a new language or rewriting the loop, put the dedupe there and nowhere later.

```
def current_threshold(self) -> float:
    load = len(self._pending_admitted)
    return self.base_threshold * (1 + self.load_sensitivity * load / self.
    capacity)

def release(self, case_id: str) -> None:
    self._pending_admitted.discard(case_id)
```

`release()` is not optional plumbing you can skip in a first integration. §6's second entry is what happens when you do.

3.3 The closed-form helper

```
def min_ticks_to_admission(threshold, tick_cap, decay_rate) -> float:
    ceiling_fraction = threshold * decay_rate / tick_cap
    if ceiling_fraction >= 1.0:
        return math.inf
    return math.log(1 - ceiling_fraction) / math.log(1 - decay_rate)
```

This function is not a side utility; it is Proposition 2 of the companion paper made executable, and it is used twice: once to *design* `base_threshold` against a target attacker cost, and once inside the adversarial test suite to check that the simulated gate actually admits a saturated attacker at the tick the closed form predicts. If you change any of `decay_rate`, `tick_cap`, or `base_threshold`, re-run this function before re-running the simulation — if the two disagree

by more than a tick, something in the implementation has drifted from the model it is supposed to realize.

4 Tuning procedure

Do not hand-tune the seven constants independently. Follow this order:

1. **Fix ρ from your tick granularity.** ρ sets the half-life of unreinforced pressure, $\ln 2/\rho$ ticks. Pick a tick length matching how often your system can plausibly observe reinforcement events (minutes, not milliseconds, for a human-moderation pipeline), then pick ρ so the half-life matches how long a genuinely urgent dispute should stay "warm" between independent reinforcements.

2. **Fix γ and δ from your identity-diversity ceiling.** $\gamma = 1$ (a single fresh source per tick fully saturates the cap) is the conservative default. δ should be small enough that your expected same-cohort repeat rate cannot reach the threshold even given unlimited time — check this with Proposition 1: the ceiling is $\delta\gamma/\rho$, and it must sit below θ_0 by a margin you are comfortable with, not just barely under it.

3. **Fix θ_0 from the calibration tension, explicitly.** Compute $x = \theta_0\rho/\gamma$ for candidate values of θ_0 and, for each, compute both (a) the genuine-case admission probability a moderately corroborated real dispute in your population is likely to clear, and (b) $n_{\min}(x)$ from `min_ticks_to_admission`. Pick the value where (b) is large enough to matter operationally (multiple sustained rounds of attacker effort, not one) while (a) is still comfortably clearable by your real traffic. There is no value that maximizes both; Corollary 1 proves that formally, so do not spend time looking for one.

4. **Fix κ and C last, against your actual expensive-tier capacity.** C should be your real concurrent capacity for the expensive stage (however many cases it can process at once without degrading), and κ controls how sharply the bar rises as that capacity fills. Start at $\kappa = 0.5$ and increase it if the ablation test (§4, and the paper's Attack 5) does not spread concurrent admissions out enough for your operational tolerance.

5. **Re-run the adversarial test suite with your new constants, not just the sanity control.** A tuning that only checks that genuine cases still get admitted, without re-running the burst and sustained-flood attacks, has not actually been validated.

5 Integration checklist

☐ Confirmed the admission boundary you are protecting matches the three conditions of §1 — in particular, that "admission" is currently a single-signal, single-tick decision, not something already rate-limited some other way.

☐ Identified your real tick unit and your real reinforcement-event source (what, concretely, counts as "a cohort reinforcing a case" in your system — a report, a rebuttal, a resubmission).

☐ Chosen a cohort-identity proxy and written down its honest limitation. This prototype uses a structural `rater_creation_cohort` id; whatever proxy you use, the gate's independence discount is only as good as that proxy's resistance to spoofing (§8).

☐ Wired `release()` into whatever event marks a case as finished in the expensive tier — not into the admission event, and not left uncalled (§6, second entry).

- ☐ Re-derived $x = \theta_0\rho/\gamma$ for your own genuine-case reinforcement rate rather than copying the reference constants verbatim (§4).
- ☐ Re-run all six tests in `tests/test_admission_control.py` against your constants, and read the printed numbers, not just the pass/fail status — the sanity-control admission rate and the measured attacker delay are the numbers that tell you whether the tuning is actually workable for your traffic.
- ☐ Decided, explicitly, what happens to a case that never crosses the threshold. This gate has no forced-timeout escape hatch of its own; if your system needs one (a human ombudsman review after N ticks of stalled pressure, say), it is a separate mechanism layered on top, not a hidden default here.

6 Troubleshooting

Both rows below are real bugs caught while building and adversarially testing this gate, not hypothetical failure modes.

Symptom	Root cause	Fix
A same-cohort flood still reaches the tick cap and gets admitted, even though the repeat discount is configured correctly	Same-cohort contributions were summed *before* deduplication within a tick, so enough discounted entries from one source could still add up to the cap through sheer in-tick volume	Dedupe cohorts per tick with a plain `set()` call *before* applying the discount and the cap, not after
Under a sustained multi-campaign attack simulation, the admission threshold climbs steadily and later, genuinely well-corroborated cases stop getting admitted at all	`release()` was never called, so `_pending_admitted` only ever grows, and `current_threshold()` rises monotonically as a pure artifact of the simulation, not a real property of the mechanism	Model a bounded expensive-tier dwell time and call `release()` when it elapses; do not loosen the test's assertion to paper over a threshold that should have come back down

The general lesson behind both rows: everything this gate does is sensitive to *when*, exactly, a mutating step runs relative to the cap or the load signal it feeds. Whenever you touch the tick loop, re-check the ordering, not just the arithmetic.

7 Adapting to a different admission boundary

1. **Identify your cohort proxy.** Anything that plausibly distinguishes independent sources works: account age plus creation cluster, network-origin fingerprint, or (weakest, but sometimes all you have) plain user id. The gate's independence discount is only as strong as this proxy's resistance to being spoofed fresh every tick — state that limitation explicitly wherever you deploy this, as §8 does here.
2. **Identify your load signal.** It must be a real, currently-occupied count that drains as cases leave the expensive tier, not a cumulative counter. If your expensive tier has no natural "currently occupying" notion (e.g. a one-shot expensive computation rather than a tier with dwell time), the load-adaptive threshold layer (§2, rule 3) may not transfer; the decay-plus-cap layer (rules 1–2) still does, on its own.

3. **Re-derive the calibration tension for your own genuine-case distribution.** Do not reuse $\theta_0 = 4.0$ from a different domain's reinforcement pattern; re-run the procedure of §4 against your own traffic.

4. **Re-run the adversarial suite, adapted to your domain's attack shapes.** The five scenarios in the companion paper (single-tick burst, persistent same-cohort flood, persistent cohort-spoofed flood, spray across many cases, synchronized parallel campaigns) are a reasonable starting checklist, not a closed list — think about what a resource-exhaustion attacker in your specific domain would actually try first.

8 Known limits

The gate does not close the sustained, identity-diverse attack; it delays it, by a factor that is itself a tunable trade-off against the bar for genuine cases (Corollary 1 of the companion paper). It assumes the cohort-identity proxy has some real cost to spoof; if an attacker can generate a structurally fresh-looking identity at zero marginal cost every tick, the delay factor is the entire defense, not a backstop on top of something stronger. The load-adaptive layer assumes the load signal is wired to real occupancy, per the second troubleshooting row above — get that wrong and the layer's behavior is undefined by this document. And the gate is an admission control, not a content-judgment mechanism: it says nothing about whether a case, once admitted, is decided correctly by the expensive stage it gates access to.

For the formal model, the two propositions and their proofs, the calibration-tension corollary, and the prior-art survey, see the companion paper in this package.